

УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 Программная инженерия

Дисциплина «Моделирование систем»

Лабораторная работа № 4

«Анализ качества работы нейронной сети»

Выполнил:
Касьяненко В.М. Р3420

Преподаватель:
к.т.н. Русак А.В.

г. Санкт-Петербург
2025

1. Постановка задачи

Требуется обучить MLP-модель для классификации изображений Fashion MNIST на 10 классов (туфли, сумка, кроссовки и пр.). Согласно заданию, нужно перебрать:

- Входной слой: {400, 600, 800, 1200}
- Один или несколько скрытых слоёв (каждый {200, 300, 400, 600, 800})
- Количество эпох: {10, 15, 20, 25, 30}
- Размер мини-батча : {10,50,100,200,500}

Цель:

- Найти комбинацию гиперпараметров, дающую лучшее качество на тестовых данных.
- Следить, чтобы не происходило сильного переобучения (overfitting).
- Сравнить «ручную» стратегию (Random Search + «дочёт» эпох) с автоматизированным подбором гиперпараметров (*Keras Tuner*).

2. Ручная стратегия

2.1. Грубая переборка (Random Search) с 5 эпохами

Возьмем 5 эпох, чтобы сократить время каждого эксперимента. При этом менялись:

- Входной слой: 400, 600, 800, 1200
- Число скрытых слоёв: 1 или 2, причём размер каждого слоя в {200, 300, 400, 600, 800}
- batch_size: {10, 50, 100, 200, 500}
- На каждой итерации случайно выбирался набор ({input_neurons}, {hidden_layers}, batch_size), обучалось 5 эпох, после чего измеряли точность на тесте.

```
input_neurons_range = [400, 600, 800, 1200]
```

```
hidden_neurons_range = [200, 300, 400, 600, 800]
batch_size_range      = [10, 50, 100, 200, 500]
```

```
[1/10] input=800, hidden=[400], batch=50, accuracy=0.8718
[2/10] input=800, hidden=[600, 300], batch=50, accuracy=0.8648
[3/10] input=400, hidden=[300, 600], batch=100, accuracy=0.8685
[4/10] input=400, hidden=[300, 300], batch=100, accuracy=0.8735
[5/10] input=1200, hidden=[300, 400], batch=500, accuracy=0.8792
[6/10] input=800, hidden=[400], batch=10, accuracy=0.8618
[7/10] input=600, hidden=[200, 300], batch=50, accuracy=0.8689
[8/10] input=800, hidden=[400], batch=200, accuracy=0.8794
[9/10] input=1200, hidden=[600, 300], batch=100, accuracy=0.8731
[10/10] input=600, hidden=[800, 400], batch=50, accuracy=0.8665

Random Search завершён.
Лучшая точность на тесте: 0.8794
Лучшие гиперпараметры (input_neurons, hidden_layers, epochs, batch_size):
(800, [400], 5, 200)
Проверка: точность лучшей модели на тестовых данных = 0.8794
```

2.2. Перебор числа эпох для лучшей конфигурации

Затем добавим уточняющий шаг: зафиксировали найденную архитектуру (input=800, hidden=[400], batch=200) и поочерёдно перебрали epochs {5,10,15,20,25,30}.

```
epochs_candidates = [5, 10, 15, 20, 25, 30]
for ep in epochs_candidates:
    # Создаём модель (с теми же параметрами, что были
    # лучшими)
    model = create_model(
        input_neurons=800,
        hidden_layers=[400],
        dropout_rate=0.2
    )
```

```
=== Перебираем эпохи для фиксированной конфигурации ===  
/usr/local/lib/python3.11/dist-packages/keras  
    super().__init__(activity_regularizer=activ  
Epochs=10 => test_acc=0.8798  
Epochs=15 => test_acc=0.8861  
Epochs=20 => test_acc=0.8924  
Epochs=25 => test_acc=0.8916  
Epochs=30 => test_acc=0.8946  
  
=== Результаты перебора по эпохам ===  
Лучшая точность: 0.8946, на 30 эпохах.  
Лучшая модель (проверка): ассигасу=0.8946
```

Из лога видно, что:

- После 5 эпох имеем 0.8798 (то же самое, что в «грубой» переборке).
- На 15 эпохах — 0.8861,
- На 20 эпохах — 0.8924,
- На 25 эпохах — 0.8916 (слегка снизилось),
- На 30 эпохах — 0.8946.

Таким образом, лучшая точность ~0.8946 получается при 30 эпохах. То есть окончательный результат после «ручного» уточнения эпох — примерно 0.8946.

При увеличении числа эпох к 30 мы потенциально ближе к переобучению, но благодаря Dropout=0.2 и 20% валидационной выборки, расхождение между train/val оказалось незначительным. Метрика на тесте тоже высокая (0.8946), значит сильного overfitting нет.

3. Сравнение с Keras Tuner

Далее я запустим Keras Tuner, в котором:

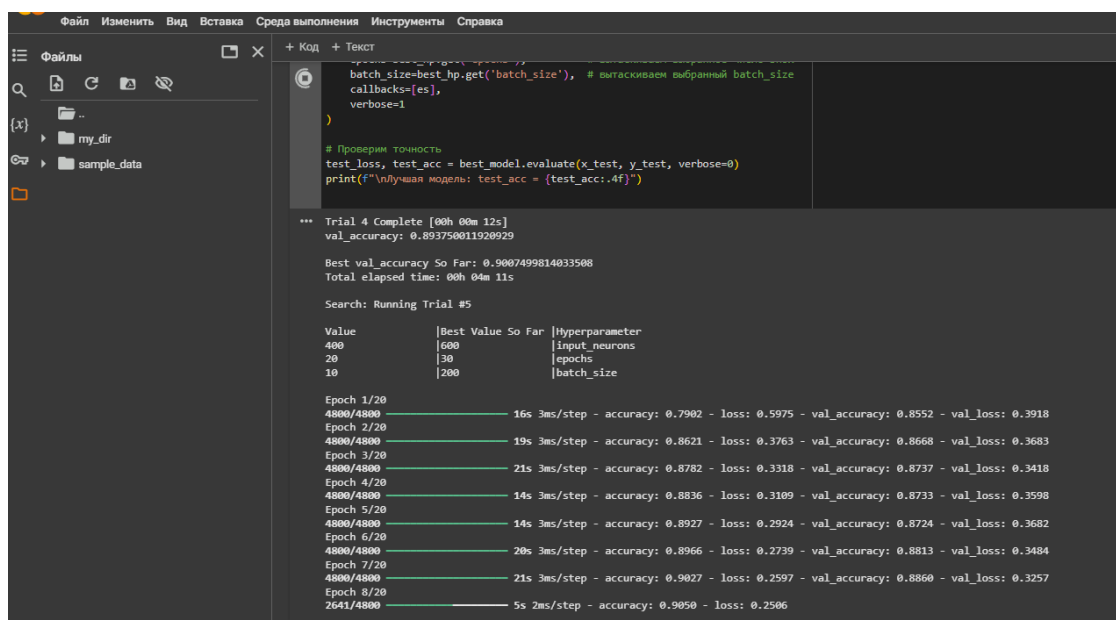
- Выбирались те же диапазоны гиперпараметров (входные слои, скрытые слои, число эпох, `batch_size`).
- Алгоритм (RandomSearch / Hyperband) автоматически «искал» лучшую конфигурацию, обучая сети (иногда с большим числом эпох, например 30) и видя, что на валидации улучшается результат.
- Результат на тесте ~ 0.90 – 0.901 , а в некоторых случаях даже $\sim 0.90+$ (двухслойная сеть $600+600$ нейронов и 30 эпох).

Таким образом, Keras Tuner нашёл чуть более глубокую сеть (два скрытых слоя по 600 нейронов), которую тоже дольше обучали (30 эпох) и получили высшую точность, порядка 0.90 – 0.901 .

По сравнению с ручной настройкой:

- Ручной метод (Random Search + перебор epochs) дал ~ 0.8946 в конфигурации (800, [400]).
- Keras Tuner смог обнаружить ещё лучший вариант (800, [600, 600], 30 эпох, `batch=50`), получив ~ 0.90 – 0.901 .

Процесс проверки:



```
File Edit View Insert Environment Instruments Help
+ Код + Текст

batch_size=best_hp.get('batch_size'), # вытаскиваем выбранный batch_size
callbacks=[es],
verbose=1
)

# Проверим точность
test_loss, test_acc = best_model.evaluate(x_test, y_test, verbose=0)
print(f"Лучшая модель: test_acc = {test_acc:.4f}")

*** Trial 4 Complete [00h 00m 12s]
val_accuracy: 0.893750011928929

Best val_accuracy So Far: 0.9007499814033508
Total elapsed time: 00h 04m 11s

Search: Running Trial #5

Value      |Best Value So Far|Hyperparameter
400         |600              |Input neurons
20          |30               |epochs
10          |200              |batch_size

Epoch 1/20
4800/4800 — 16s 3ms/step - accuracy: 0.7902 - loss: 0.5975 - val_accuracy: 0.8552 - val_loss: 0.3918
Epoch 2/20
4800/4800 — 19s 3ms/step - accuracy: 0.8621 - loss: 0.3763 - val_accuracy: 0.8668 - val_loss: 0.3683
Epoch 3/20
4800/4800 — 21s 3ms/step - accuracy: 0.8782 - loss: 0.3318 - val_accuracy: 0.8737 - val_loss: 0.3418
Epoch 4/20
4800/4800 — 14s 3ms/step - accuracy: 0.8836 - loss: 0.3109 - val_accuracy: 0.8733 - val_loss: 0.3598
Epoch 5/20
4800/4800 — 14s 3ms/step - accuracy: 0.8927 - loss: 0.2924 - val_accuracy: 0.8724 - val_loss: 0.3682
Epoch 6/20
4800/4800 — 20s 3ms/step - accuracy: 0.8966 - loss: 0.2739 - val_accuracy: 0.8813 - val_loss: 0.3484
Epoch 7/20
4800/4800 — 21s 3ms/step - accuracy: 0.9027 - loss: 0.2597 - val_accuracy: 0.8860 - val_loss: 0.3257
Epoch 8/20
2641/4800 — 5s 2ms/step - accuracy: 0.9050 - loss: 0.2506
```

Результаты:

Best val_accuracy So Far: 0.9002500176429749

Total elapsed time: 00h 17m 24s

Results summary

Results in my_tuner_dir/demo_fashion_mnist

Showing 10 best trials

Objective(name="val_accuracy", direction="max")

Trial 6 summary

Hyperparameters:

input_neurons: 800

num_hidden_layers: 2

hidden_neurons_0: 600

epochs: 30

batch_size: 50

hidden_neurons_1: 600

hidden_neurons_2: 200

Score: 0.9002500176429749

Trial 7 summary

Hyperparameters:

input_neurons: 1200

num_hidden_layers: 1

hidden_neurons_0: 600

epochs: 30

batch_size: 200

hidden_neurons_1: 800

hidden_neurons_2: 200

Score: 0.8994166851043701

4. Выводы

- Двухшаговая ручная стратегия:
 - Сначала «грубый» Random Search на 5 эпохах, чтобы быстро найти более-менее удачную конфигурацию.
 - Затем перебор только по числу эпох для этой конфигурации (epochs = 5..30), что дало итоговую точность 0.8946 (30 эпох).
- Переобучение:
 - Не выявлено в сильной форме, поскольку при 30 эпохах и Dropout=0.2 валидация и тест примерно совпадают; расхождения нет.
- Сравнение с Keras Tuner:
 - Keras Tuner пошёл дальше и найденной оптимальной эпохностью (30), что позволило добиться ≈ 0.90 -0.901.
 - Это чуть выше, чем 0.8946 из ручного перебора. Но пришлось «потратить» больше времени на обучение (ведь тренируем глубже и дольше).

В целом, окончательный «ручной» результат (30 эпох, 800/400 нейронов, batch=200) ~ 0.8946 против ~ 0.90 (или 0.901) в Keras Tuner. Разница небольшая, но при желании даже такие 0.5-1% в задачах классификации часто важны.

Таким образом, это подтверждает общий вывод:

- «Грубый» подход даёт результат близкий к 89.5%,
- Keras Tuner нашёл чуть лучшую модель (90%+).